ALMATY TECHNOLOGICAL UNIVERSITY

СРО№1

Тема: Реестр учебных проектов

Группа: ИС 24-21
Студент: 3 курса
ФИО: Шакир Мирас

Вариант №7

СОДЕРЖАНИЕ

1. Введение
2. Анализ предметной области
3. Требования к системе
4. Проектирование модели данных
5. Архитектура решения и выбор технологии
6. Реализация программного приложения
7. Запросы к базе данных
8. Индексирование и оптимизация
9. Тестирование приложения
10. Инструкция по запуску
11. Заключение
12. Список использованных источников

1. ВВЕДЕНИЕ

В современных образовательных организациях активно используются информационные системы для организации учебного процесса, выполнения групповых заданий и контроля результатов студентов. Одним из важных направлений является управление учебными проектами, в рамках которых студенты выполняют задания, взаимодействуют с руководителями и отслеживают состояние работы.

При ручном управлении проектами информация часто хранится в отдельных таблицах, документах и электронных сообщениях. Такой подход затрудняет поиск информации, контроль сроков выполнения задач и получение актуальных сведений о ходе проектов.

Для решения данной проблемы разрабатывается программная система «Реестр учебных проектов». Она предназначена для хранения сведений о пользователях, учебных проектах и задачах, а также для выполнения основных операций добавления, просмотра, изменения и удаления данных.

В качестве системы управления базами данных используется MongoDB — документоориентированная NoSQL-СУБД. Для взаимодействия с базой данных применяется язык программирования Python и библиотека PyMongo.

Цель работы — разработать прототип информационной системы для управления учебными проектами с использованием MongoDB и Python.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить предметную область управления учебными проектами.
2. Определить функциональные и нефункциональные требования.
3. Разработать модель данных для MongoDB.
4. Выбрать технологию хранения данных и обосновать её применение.
5. Реализовать основные операции CRUD.
6. Подготовить набор демонстрационных данных.
7. Реализовать запросы для поиска проектов и задач.

8. Создать индексы для ускорения типовых запросов.
9. Определить сценарии проверки работоспособности приложения.
10. Подготовить инструкцию по установке и запуску программы.

Объект исследования — процесс управления учебными проектами.

Предмет исследования — методы хранения, обработки и поиска данных в документоориентированной базе данных MongoDB.

Практическая значимость работы заключается в создании прототипа, который можно использовать как основу для дальнейшей разработки полноценной системы управления учебными проектами.

2. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1. Описание предметной области

Учебный проект представляет собой организованную деятельность одного или нескольких студентов, направленную на достижение определённого результата. Каждый проект имеет название, описание, руководителя, участников, статус и даты выполнения.

Для достижения цели проекта он разделяется на отдельные задачи. Каждая задача назначается исполнителю и имеет собственный статус, приоритет и срок выполнения.

В разрабатываемой системе выделяются три основные сущности:

Пользователь (User). Хранит информацию о студенте или руководителе проекта. Для пользователя сохраняются имя, адрес электронной почты и роль в системе.

Проект (Project). Содержит название, описание, идентификатор руководителя, список участников, статус и даты создания и завершения.

Задача (Task). Представляет отдельную работу в рамках проекта. Содержит название, описание, идентификаторы проекта и исполнителя, статус, приоритет, срок выполнения и дату создания.

Связь между проектами и задачами позволяет получать список работ конкретного проекта. Связь между пользователями и задачами позволяет определять текущую нагрузку исполнителей.

2.2. Основные бизнес-процессы

В системе предусмотрены следующие бизнес-процессы:

1. Регистрация пользователей.

2. Создание учебного проекта.
3. Добавление участников в проект.
4. Создание задач для проекта.
5. Назначение исполнителей.
6. Изменение статуса задачи.
7. Поиск проектов по статусу.
8. Получение списка задач конкретного исполнителя.
9. Поиск просроченных задач.
10. Получение сводной статистики по задачам проекта.

Эти процессы позволяют организовать хранение информации и обеспечить доступ к наиболее востребованным сведениям.

2.3. Актуальность разработки

Актуальность проекта обусловлена необходимостью автоматизации контроля учебной деятельности. Централизованное хранение данных позволяет уменьшить количество ошибок при ведении реестра, упростить поиск информации и ускорить получение отчётных данных.

Использование MongoDB позволяет хранить связанные по смыслу документы с гибкой структурой. При необходимости в модель можно добавить новые поля, например отметку о выполнении, комментарии, вложения или историю изменений статуса.

3. ТРЕБОВАНИЯ К СИСТЕМЕ

3.1. Функциональные требования

Функциональные требования описывают действия, которые должна выполнять система.

Код	Требование
FR-01	Система должна обеспечивать подключение к серверу MongoDB.
FR-02	Система должна хранить сведения о пользователях.
FR-03	Система должна создавать новые учебные проекты.
FR-04	Система должна создавать задачи в рамках существующих проектов.
FR-05	Система должна назначать исполнителей задач.
FR-06	Система должна изменять статус задачи.
FR-07	Система должна удалять выбранные задачи.
FR-08	Система должна выводить список всех проектов.
FR-09	Система должна находить задачи по исполнителю и проекту.
FR-10	Система должна находить просроченные задачи и формировать сводную статистику.

3.2. Нефункциональные требования

Нефункциональные требования определяют характеристики качества системы.

Код	Требование
NFR-01	Приложение должно запускаться в среде Python.
NFR-02	Доступ к данным должен выполняться через MongoDB и библиотеку PyMongo.
NFR-03	Для ускорения поиска должны применяться индексы.
NFR-04	Система должна выводить понятные сообщения при ошибках подключения и выполнения операций.
NFR-05	Структура приложения должна допускать дальнейшее расширение функциональности.

3.3. Access Patterns — типовые сценарии доступа к данным

Access Patterns описывают, каким образом приложение обращается к базе данных. Их определение необходимо для выбора структуры документов и индексов.

№	Сценарий доступа	Приоритет	Ожидаемый результат
AP-01	Получение всех проектов	Высокий	Список проектов с основными сведениями
AP-02	Поиск проектов по статусу	Высокий	Проекты, соответствующие выбранному статусу
AP-03	Получение участников проекта	Средний	Пользователи, связанные с проектом
AP-04	Получение задач исполнителя	Высокий	Все задачи указанного пользователя
AP-05	Получение задач проекта	Высокий	Задачи, относящиеся к выбранному проекту
AP-06	Поиск открытых задач высокого приоритета	Высокий	Незавершённые задачи с высоким приоритетом
AP-07	Поиск просроченных задач	Высокий	Незавершённые задачи с истёкшим сроком
AP-08	Подсчёт задач по статусам	Средний	Количество задач каждого статуса

Таким образом, определены восемь основных сценариев доступа. Они используются при проектировании запросов и индексов.

4. ПРОЕКТИРОВАНИЕ МОДЕЛИ ДАННЫХ

4.1. Выбор документоориентированно

й модели

MongoDB хранит данные в виде BSON-документов, структура которых близка к JSON. Документы объединяются в коллекции.

В отличие от реляционной модели, MongoDB не требует, чтобы все документы коллекции имели строго одинаковый набор полей. Это удобно для учебных проектов, поскольку их характеристики могут различаться.

В разрабатываемой системе используется база данных `student_project_registry`, состоящая из трёх коллекций:

- `users`;
- `projects`;
- `tasks`.

Для связывания документов применяются идентификаторы `ObjectId`. Например, задача хранит идентификатор проекта и идентификатор исполнителя.

4.2. Коллекция users

Коллекция `users` предназначена для хранения пользователей системы.

Пример документа:

```
{
  "_id": "ObjectId",
  "name": "Иван Петров",
  "email": "ivan@example.com",
  "role": "student"
}
```

Основные поля:

Поле	Тип данных	Назначение
`_id`	ObjectId	Уникальный идентификатор пользователя
`name`	String	Имя пользователя
`email`	String	Электронная почта
`role`	String	Роль пользователя

Поле `email` должно быть уникальным. Для этого создаётся уникальный индекс.

4.3. Коллекция projects

Коллекция `projects` содержит сведения об учебных проектах.

Пример документа:

```
{
  "_id": "ObjectId",
  "title": "Разработка информационной системы",
  "description": "Учебный проект по разработке
приложения",
  "owner_id": "ObjectId",
  "member_ids": ["ObjectId"],
  "status": "active",
  "created_at": "2026-10-20T10:00:00Z",
  "due_date": "2026-12-20T00:00:00Z"
}
```

Основные поля:

Поле	Тип данных	Назначение
`_id`	ObjectId	Идентификатор проекта
`title`	String	Название проекта
`description`	String	Описание проекта
`owner_id`	ObjectId	Идентификатор руководителя
`member_ids`	Array	Список идентификаторов участников
`status`	String	Состояние проекта
`created_at`	Date	Дата создания
`due_date`	Date	Планируемая дата завершения

В качестве статусов проекта могут использоваться значения `active`, `completed` и `planned`.

4.4. Коллекция tasks

Коллекция `tasks` предназначена для хранения задач.

Пример документа:

```
{
  "_id": "ObjectId",
  "project_id": "ObjectId",
  "title": "Подготовить модель данных",
  "description": "Описать коллекции и связи",
  "assignee_id": "ObjectId",
  "status": "in_progress",
  "priority": "high",
  "due_date": "2026-11-10T00:00:00Z",
  "created_at": "2026-10-20T10:00:00Z"
}
```

Основные поля:

Поле	Тип данных	Назначение
`_id`	ObjectId	Идентификатор задачи
`project_id`	ObjectId	Идентификатор проекта
`title`	String	Название задачи
`description`	String	Описание задачи
`assignee_id`	ObjectId	Идентификатор исполнителя
`status`	String	Статус выполнения
`priority`	String	Приоритет
`due_date`	Date	Срок выполнения
`created_at`	Date	Дата создания

Для задач используются статусы `todo`, `in_progress` и `done`. Приоритет может принимать значения `low`, `medium` и `high`.

В рабочем коде даты должны сохраняться как объекты `datetime`, которые PyMongo преобразует в BSON Date, а не как обычные текстовые строки.

4.5. Связи между сущностями

Связи между коллекциями организуются через идентификаторы.

- Один пользователь может руководить несколькими проектами.
- Один проект может включать нескольких участников.
- Один проект может содержать множество задач.
- Один пользователь может быть исполнителем нескольких задач.

Текстовая схема модели данных:

```
users
  |
  | _id
  |
  +------ projects.owner_id
  |
  +------ projects.member_ids
  |
  +------ tasks.assignee_id

projects
  |
  | _id
  |
  +------ tasks.project_id
```

Такая модель уменьшает дублирование данных пользователей и проектов. Информация о задаче хранится отдельно, что позволяет эффективно получать задачи по исполнителю и проекту.

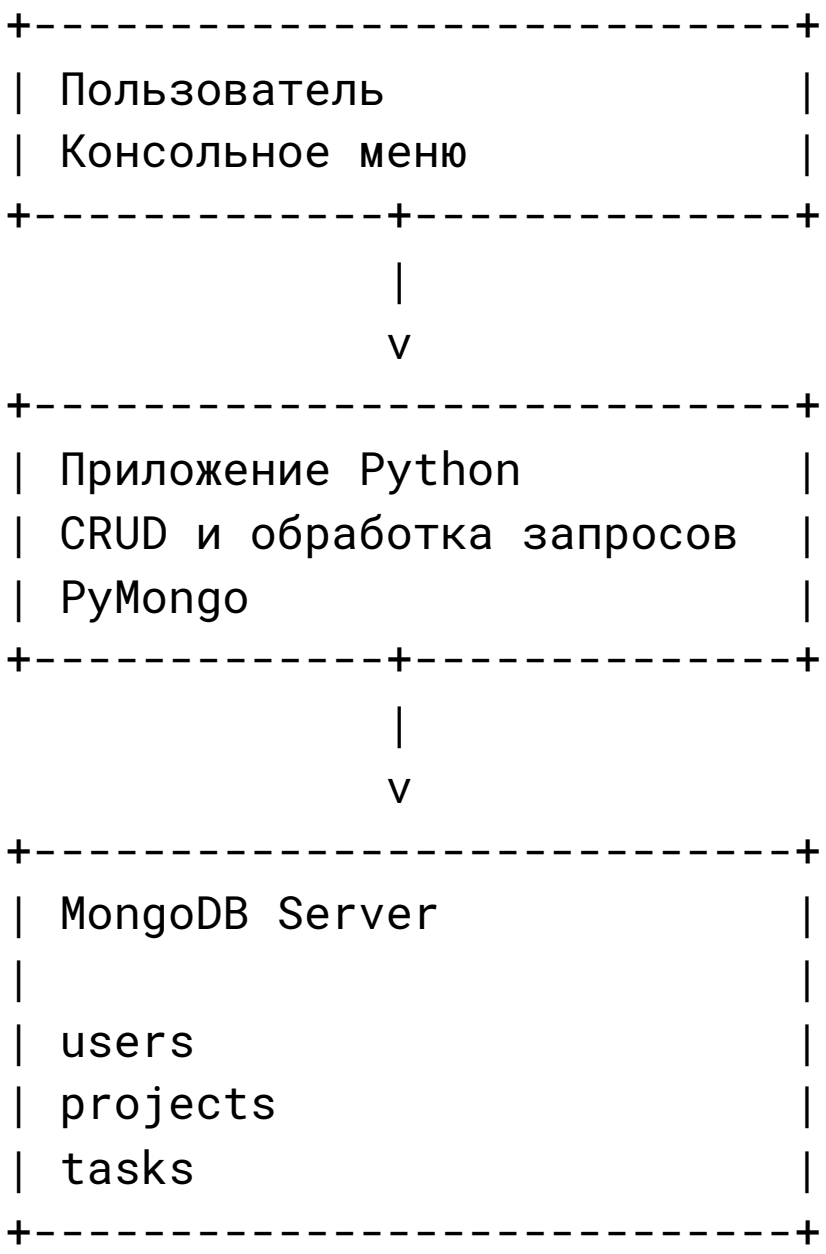
5. АРХИТЕКТУРА РЕШЕНИЯ И ВЫБОР ТЕХНОЛОГИИ

5.1. Архитектура приложения

Разрабатываемая система имеет простую трёхкомпонентную архитектуру:

1. Пользователь взаимодействует с консольным интерфейсом.
2. Программная логика на Python обрабатывает команды и формирует запросы.
3. MongoDB хранит данные и возвращает результаты запросов.

Архитектурная схема:



Консольный интерфейс позволяет выполнять основные операции без разработки отдельного графического приложения.

5.2. Сравнение альтернатив

При проектировании были рассмотрены несколько NoSQL-технологий.

Критерий	MongoDB	Redis	Cassandra	Neo4j
Основная модель	Документы	Ключ-значение	Широкие столбцы	Граф
Гибкая структура данных	Да	Ограниченно	Да, с особенностям и модели	Да
Удобство хранения проектов и задач	Высокое	Среднее	Среднее	Высокое для сложных связей
Поиск по полям документов	Да	Не является основной сильной стороной	Зависит от модели и ключей	Да, через графовые запросы
Группировка и агрегация	Есть Aggregation Pipeline	Ограниченно	Возможности зависят от решения	Есть графовые запросы
Основное применение	Документные приложения	Кэширование и быстрые операции	Распределённые большие нагрузки	Системы со сложными связями

Для данного проекта MongoDB является подходящим решением, поскольку позволяет хранить проекты и задачи в отдельных коллекциях, поддерживает фильтрацию документов, индексы и агрегирующие запросы.

Redis больше ориентирован на быстрый доступ к данным по ключам и кэширование. Cassandra предназначена для распределённых систем с высокой нагрузкой и требует тщательного проектирования запросов. Neo4j особенно полезна, когда основное значение имеют сложные взаимосвязи между объектами.

5.3. ADR-001: выбор MongoDB

Идентификатор решения: ADR-001.

Название: Выбор MongoDB в качестве основной СУБД.

Статус: принято.

Контекст: система должна хранить сведения о пользователях, проектах и задачах, обеспечивать CRUD-операции и выполнять выборки по нескольким полям.

Рассмотренные альтернативы: MongoDB, Redis, Cassandra, Neo4j.

Решение: использовать MongoDB совместно с Python и PyMongo.

Причины выбора:

- документоориентированная модель соответствует структуре предметной области;
- поддерживаются фильтрация, сортировка и агрегация;
- доступны индексы по отдельным полям и их комбинациям;
- PyMongo обеспечивает удобное взаимодействие с базой из Python;
- систему можно расширять добавлением новых полей и коллекций.

Последствия решения: необходимо установить и запустить MongoDB Server либо использовать удалённый экземпляр MongoDB. Также требуется контролировать корректность ссылочных идентификаторов и структуру документов.

6. РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРИЛОЖЕНИЯ

6.1. Используемые средства

Для реализации проекта используются следующие технологии:

- Python — язык программирования;
- MongoDB — документоориентированная система управления базами данных;
- PyMongo — библиотека для работы с MongoDB из Python;
- Visual Studio Code — среда разработки.

Библиотека PyMongo обеспечивает подключение к серверу, выполнение запросов, добавление документов, обновление и удаление записей.

Важно отметить, что PyMongo не является самой СУБД. Для подключения к локальному адресу `mongodb://localhost:27017` должен быть установлен и запущен сервер MongoDB.

6.2. Подключение к базе данных

В программе задаются параметры подключения:

```
MONGO_URI = "mongodb://localhost:27017"
DB_NAME = "student_project_registry"
```

Затем создаётся клиент MongoDB и выбирается база данных:

```
from pymongo import MongoClient

client = MongoClient(
    "mongodb://localhost:27017",
    serverSelectionTimeoutMS=5000
)

client.admin.command("ping")

db = client["student_project_registry"]
```

Команда `ping` позволяет проверить доступность сервера. Если соединение не устанавливается, необходимо проверить установку MongoDB Server, его состояние и адрес подключения.

6.3. Создание коллекций и индексов

Для повышения производительности создаются индексы:

```
db.projects.create_index("status")
db.projects.create_index("owner_id")

db.users.create_index(
    "email",
    unique=True
)

db.tasks.create_index([
    ("assignee_id", 1),
    ("status", 1)
])

db.tasks.create_index([
    ("project_id", 1),
    ("status", 1)
])

db.tasks.create_index([
    ("status", 1),
    ("due_date", 1)
])
```

Индексы позволяют быстрее находить документы по часто используемым полям. Уникальный индекс электронной почты предотвращает сохранение двух пользователей с одинаковым адресом электронной почты.

Фактическая эффективность индексов зависит от объёма данных, распределения значений и конкретных запросов. Для проверки производительности можно использовать `explain()`.

6.4. Наполнение базы демонстрационными данными

Для демонстрации работы приложения используется функция `seed_database(db)`.

В проекте предусмотрено создание следующих синтетических данных:

Коллекция	Количество документов
users	10
projects	5
tasks	30
Всего	**45**

Данные создаются автоматически при выполнении программы, если коллекция `projects` ещё не содержит документов.

Синтетические данные позволяют продемонстрировать поиск проектов, получение задач исполнителя, поиск просроченных работ и агрегацию по статусам.

Важная особенность текущей реализации: проверка наполнения выполняется по коллекции `projects`. Если проекты уже существуют, но коллекции `users` или `tasks` пусты либо заполнены частично, автоматическое заполнение может не восстановить их содержимое. Для повторных запусков и более надёжной инициализации эту логику можно улучшить.

6.5. Реализация CRUD-операций

CRUD — это четыре основные операции управления данными:

- Create — создание;
- Read — чтение;
- Update — изменение;
- Delete — удаление.

В программе предусмотрены функции создания пользователей, проектов и задач, получения документов, изменения статуса задачи и удаления задачи.

Пример создания пользователя:

```
def create_user(db, name, email, role):
    user = {
        "name": name,
        "email": email,
        "role": role
    }

    result = db.users.insert_one(user)
    return result.inserted_id
```

Пример изменения статуса задачи:

```
def update_task_status(db, task_id, new_status):
    result = db.tasks.update_one(
        {"_id": task_id},
        {"$set": {"status": new_status}}
    )

    return result.modified_count
```

Пример удаления задачи:

```
def delete_task(db, task_id):
    result = db.tasks.delete_one(
        {"_id": task_id}
    )

    return result.deleted_count
```

Эти фрагменты показывают общий принцип работы с коллекцией. При использовании в полноценном приложении необходимо также проверять существование документов, допустимые значения статусов и ошибки базы данных.

6.6. Консольное меню

В приложении предусмотрено меню для выбора доступных операций. Пользователь может просматривать проекты, искать задачи, создавать новые записи, изменять статус и удалять задачи.

Консольное меню упрощает демонстрацию проекта на защите и позволяет проверить работу базы данных без графического интерфейса.

7. ЗАПРОСЫ К БАЗЕ ДАННЫХ

В проекте предусмотрено восемь демонстрационных запросов. Они соответствуют типовым сценариям доступа к данным, определённым на этапе проектирования.

7.1. Запрос 1. Получение всех проектов

```
projects = list(db.projects.find({}))
```

Запрос возвращает все документы коллекции `projects`.

Применение: просмотр полного реестра учебных проектов.

7.2. Запрос 2. Поиск активных проектов

```
projects = list(
    db.projects.find({"status": "active"})
)
```

Запрос возвращает проекты, статус которых равен `active`.

Применение: получение списка проектов, над которыми ведётся работа.

7.3. Запрос 3. Получение участников проекта

```
project = db.projects.find_one(
    {"_id": project_id}
)

if project:
    members = list(
        db.users.find({
            "_id": {"$in":
project.get("member_ids", [])}
        })
    )
```

Сначала находится проект, затем по списку идентификаторов участников извлекаются соответствующие пользователи.

Применение: отображение участников выбранного проекта.

7.4. Запрос 4. Получение задач конкретного исполнителя

```
tasks = list(
    db.tasks.find({
        "assignee_id": user_id
    })
)
```

Запрос возвращает задачи, назначенные определённому пользователю.

Применение: контроль индивидуальной нагрузки студента.

7.5. Запрос 5. Получение задач выбранного проекта

```
tasks = list(
    db.tasks.find({
        "project_id": project_id
    })
)
```

Запрос позволяет получить полный список задач конкретного проекта.

Применение: контроль состояния работ по проекту.

7.6. Запрос 6. Открытые задачи высокого приоритета

```
tasks = list(
    db.tasks.find({
        "status": {"$ne": "done"},
        "priority": "high"
    })
)
```

Запрос выбирает задачи высокого приоритета, которые ещё не имеют статуса done.

Применение: определение наиболее важных незавершённых работ.

Этот запрос сложнее обычного поиска по одному полю, поскольку использует условия по приоритету и статусу.

7.7. Запрос 7. Поиск просроченных незавершённых задач

```
from datetime import datetime, timezone

now = datetime.now(timezone.utc)

tasks = list(
    db.tasks.find({
        "due_date": {"$lt": now},
        "status": {"$ne": "done"}
    })
)
```

Запрос возвращает задачи, срок выполнения которых уже прошёл, а статус не равен `done`.

Применение: выявление просроченных работ и контроль соблюдения сроков.

Результат зависит от фактических дат в базе данных. Если просроченных задач нет, запрос может вернуть пустой список.

7.8. Запрос 8. Агрегация задач по статусам проекта

```
pipeline = [
    {"$match": {"project_id": project_id}},
    {
        "$group": {
            "_id": "$status",
            "count": {"$sum": 1}
        }
    },
    {"$sort": {"count": -1}}
]

result = list(
    db.tasks.aggregate(pipeline)
)
```

Агрегация сначала выбирает задачи проекта, затем группирует их по статусу и подсчитывает количество документов в каждой группе.

Применение: получение сводной статистики о ходе выполнения проекта.

Данный запрос использует Aggregation Pipeline — механизм обработки данных в MongoDB.

7.9. Итоги реализации запросов

Реализованные запросы обеспечивают получение списка проектов, поиск по статусам, выборку задач по исполнителю и проекту, выявление просроченных работ и формирование сводной статистики.

Два наиболее сложных сценария — поиск просроченных задач и агрегация задач по статусам. Они демонстрируют применение операторов сравнения, логических условий и этапов агрегации.

8. ИНДЕКСИРОВАНИЕ И ОПТИМИЗАЦИЯ

8.1. Назначение индексов

Индекс представляет собой структуру данных, которая помогает MongoDB находить документы без полного перебора коллекции в подходящих случаях.

В проекте используются следующие индексы:

Коллекция	Поля индекса	Назначение
`users`	`email` — уникальный	Поиск пользователя по электронной почте и обеспечение уникальности
`projects`	`status`	Поиск проектов по статусу
`projects`	`owner_id`	Поиск проектов руководителя
`tasks`	`assignee_id`, `status`	Выборка задач исполнителя с учётом статуса
`tasks`	`project_id`, `status`	Выборка задач проекта по статусу
`tasks`	`status`, `due_date`	Поддержка выборок по статусу и сроку выполнения

Составные индексы включают несколько полей. Порядок полей важен: индекс наиболее полезен для запросов, которые используют его начальные поля, с учётом структуры конкретного запроса.

8.2. Ожидаемый эффект оптимизации

Индексы должны сократить объём документов, просматриваемых при выполнении соответствующих запросов. Однако наличие индекса не гарантирует ускорение каждой операции.

Индексы также требуют дополнительного места и могут увеличивать затраты при добавлении и изменении документов.

Для практической оценки производительности следует сравнить планы выполнения запросов до и после создания индексов с помощью `explain("executionStats")`. В

рамках данной работы конкретные показатели ускорения не заявляются, поскольку они должны быть получены в ходе фактического измерения.

9. ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ

9.1. Цель тестирования

Цель тестирования — проверить подключение к MongoDB, корректность выполнения основных CRUD-операций, правильность фильтрации и работу агрегирующих запросов.

Ниже приведены сценарии проверки. Таблица содержит ожидаемые результаты, а не утверждение о том, что все проверки уже выполнены. Фактический результат следует заполнить после запуска программы.

9.2. Тестовые сценарии

ID	Сценарий	Ожидаемый результат
TC-01	Подключение к MongoDB	Соединение устанавливается, команда `ping` выполняется без ошибки
TC-02	Первичное заполнение базы	Создаются 10 пользователей, 5 проектов и 30 задач при выполнении условий инициализации
TC-03	Получение всех проектов	Выводится список документов из `projects`
TC-04	Поиск активных проектов	Выводятся только проекты со статусом `active`
TC-05	Получение задач исполнителя	Выводятся задачи с соответствующим `assignee_id`
TC-06	Обновление статуса задачи	Статус существующей задачи изменяется
TC-07	Удаление задачи	Указанный документ удаляется из коллекции
TC-08	Агрегация задач	Выводится количество задач по каждому статусу выбранного проекта

9.3. Пример проверки CRUD

Для проверки создания пользователя необходимо вызвать соответствующую функцию, убедиться, что она возвращает идентификатор созданного документа, а затем найти запись в коллекции.

Для проверки изменения статуса задачи необходимо выбрать существующую задачу, изменить её статус и повторно прочитать документ из базы.

Для проверки удаления задачи следует создать тестовую задачу, удалить её и убедиться, что поиск по её идентификатору больше не возвращает документ.

При проверке уникальности электронной почты необходимо попытаться добавить двух пользователей с одинаковым email. Ожидается, что уникальный индекс не позволит сохранить дубликат.

Если проверка завершилась ошибкой, следует зафиксировать её причину и при необходимости исправить программную реализацию.

9.4. Критерии успешного тестирования

Приложение можно считать прошедшим базовую функциональную проверку, если:

- подключение к серверу MongoDB работает;
- демонстрационные данные создаются в соответствии с логикой заполнения;
- запросы возвращают документы, соответствующие заданным условиям;
- изменение статуса сохраняется в базе;
- удаление действительно удаляет документ;
- уникальный индекс не допускает повторения значения email;
- агрегирующий запрос формирует ожидаемые группы и подсчёты.

Фактические результаты проверки необходимо зафиксировать после запуска приложения в используемом окружении.

10. ИНСТРУКЦИЯ ПО ЗАПУСКУ

10.1. Подготовка среды

Для запуска проекта необходимы:

- установленный Python;
- установленный и запущенный MongoDB Server либо доступный удалённый сервер MongoDB;
- библиотека PyMongo;
- редактор кода, например Visual Studio Code.

Если используется локальное подключение `mongodb://localhost:27017`, одного установленного Python недостаточно. Сервер MongoDB должен быть установлен и запущен отдельно.

10.2. Установка PyMongo

Откройте терминал в Visual Studio Code и выполните:

```
python --version
```

После проверки версии Python установите библиотеку:

```
python -m pip install pymongo
```

Если в проекте используется виртуальное окружение, сначала активируйте его, затем установите зависимость.

10.3. Проверка MongoDB

Убедитесь, что MongoDB Server установлен и запущен.

При наличии утилиты MongoDB Shell можно проверить соединение командой:

```
mongosh "mongodb://localhost:27017"
```

Если команда не распознаётся, MongoDB Shell может отсутствовать в системе. Это не всегда означает отсутствие самого сервера.

Если соединение не устанавливается, проверьте состояние службы MongoDB в Windows и корректность адреса подключения.

10.4. Запуск программы

Сохраните исходный код в файл `main.py`.

В терминале перейдите в папку проекта и выполните:

```
python main.py
```

При успешном подключении программа должна выполнить инициализацию базы данных в соответствии с предусмотренной логикой, создать необходимые индексы и показать консольное меню.

Если появляется ошибка подключения, сначала убедитесь, что сервер MongoDB работает и доступен по адресу `mongodb://localhost:27017`.

10.5. Подключение к MongoDB Atlas

В качестве альтернативы можно использовать MongoDB Atlas — облачный сервис MongoDB. В таком случае необходимо создать кластер, настроить доступ и использовать строку подключения, предоставленную сервисом.

Перед использованием удалённой базы следует изменить `MONGO_URI` в коде на соответствующую строку подключения. Учётные данные нельзя публиковать в открытом GitHub-репозитории.

Для более безопасной реализации рекомендуется читать строку подключения из переменной окружения, например `MONGODB_URI`. В текущей версии программы, если URI задан непосредственно в коде, автоматическое чтение файла `.env` не выполняется без дополнительной реализации.

11. ЗАКЛЮЧЕНИЕ

В ходе выполнения самостоятельной работы был разработан проект информационной системы «Реестр учебных проектов» на основе MongoDB и Python.

В рамках проектирования была рассмотрена предметная область, определены основные сущности и сценарии доступа к данным. Выделены три коллекции: `users`, `projects` и `tasks`. Для связи документов применяются идентификаторы MongoDB.

Сформулированы функциональные и нефункциональные требования. Определены восемь Access Patterns, отражающих наиболее востребованные операции: получение списка проектов, поиск по статусу, выборка задач исполнителя, получение задач проекта, поиск открытых задач высокого приоритета, выявление просроченных задач и формирование статистики.

Обоснован выбор MongoDB. По сравнению с альтернативными NoSQL-решениями документоориентированная модель хорошо подходит для хранения информации о проектах и задачах, а поддержка индексов и агрегирующих запросов позволяет реализовать основные требования системы.

В программной части предусмотрены операции создания пользователей, проектов и задач, изменение статуса задачи, удаление задач и выполнение демонстрационных запросов. Для тестирования подготовлены сценарии проверки подключения, CRUD-операций и агрегации. Для типовых запросов предусмотрены индексы.

Поставленная цель проектирования прототипа достигнута на уровне разработанной модели и предусмотренной программной реализации. При этом окончательное подтверждение работоспособности должно быть основано на фактическом запуске приложения и выполнении тестовых сценариев в рабочем окружении.

В дальнейшем систему можно улучшить за счёт добавления авторизации, проверки ролей пользователей, более строгой валидации данных, централизованной обработки ошибок, расширенного управления участниками проектов, автоматизированных тестов и графического интерфейса.

Таким образом, проект демонстрирует практическое применение MongoDB для хранения и обработки данных учебных проектов и показывает основные принципы проектирования NoSQL-приложений.

12. СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MongoDB Documentation. Официальная документация MongoDB. URL: https://www.mongodb.com/docs/
2. PyMongo Documentation. Официальная документация драйвера PyMongo. URL: https://pymongo.readthedocs.io/
3. MongoDB Manual: Documents. Документы в MongoDB. URL: https://www.mongodb.com/docs/manual/core/document/
4. MongoDB Manual: Indexes. Индексы в MongoDB. URL: https://www.mongodb.com/docs/manual/indexes/
5. MongoDB Manual: Aggregation Pipeline. Агрегация данных в MongoDB. URL: https://www.mongodb.com/docs/manual/core/aggregation-pipeline/
6. Python Documentation. Официальная документация Python. URL: https://docs.python.org/3/
7. Visual Studio Code Documentation. Документация среды разработки. URL: https://code.visualstudio.com/docs/

Дата обращения к электронным ресурсам: 20 октября 2026 года.